

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch24_rag

AYuSh

Contents

Chapter 24: Retrieval-Augmented Generation (RAG)	3
--	---

Chapter 24: Retrieval-Augmented Generation (RAG)

A language model's knowledge is *parametric*: everything it "knows" is baked into frozen weights at training time, which makes it lossy (facts are compressed and interpolated, not stored verbatim), stale (nothing after the training cutoff exists), unattributable (there is no source to cite), and unbounded in its confident wrongness (it will confabulate a plausible answer rather than admit ignorance). **Retrieval-augmented generation** attacks all four at once by making knowledge *non-parametric*: at query time a retriever fetches relevant text from an external corpus and places it in the model's context, so the model reasons over *evidence it is looking at* rather than memories it half-recalls. This is the dominant architecture for grounding LLMs in private, current, or citable knowledge, and it is the single most common LLM-application topic in interviews — because it is mostly a *retrieval* problem wearing a generation hat, and retrieval is a decades-old discipline with sharp, measurable trade-offs that a candidate is expected to reason about quantitatively.

This chapter is built the way the rest of the book is: every claim is a measurement produced by from-scratch code on real text (the 20 Newsgroups corpus), with embeddings from a classical LSA pipeline (TF-IDF followed by truncated SVD) so that the retrieval mechanics — not a black-box embedding API — are fully visible and reproducible. The embeddings are deliberately weak, which is a feature: it keeps absolute recall numbers modest and honest, and it exposes the *relative* effects (chunk size, similarity metric, index approximation, fusion, reranking) that survive regardless of embedder quality and that are the actual subject of interview questions. Where a technique helps it is shown helping; where it *hurts* on this corpus — pseudo-relevance feedback does — that is reported straight, because knowing the precondition a technique needs is worth more than a folk rule that it always works.

The measurements: dense retrieval recall climbing from 0.04 at rank 1 to 0.37 at rank 50, the recall/rank curve that governs how much you must retrieve (Listing 1); chunking where a *lexical* query is found at essentially any chunk size (~ 0.97) while a *semantic* query's recall rises with chunk size ($0.09 \rightarrow 0.65$) and a naive fixed-width cut loses two points of recall to boundary damage versus sentence-aware packing (Listing 2); the dot-product length bias, where an un-normalized dot product's tendency to retrieve high-norm passages correlates $+0.50$ with passage norm versus $+0.05$ for cosine (Listing 3); approximate-nearest-neighbour indexes from scratch, an IVF reaching 0.91 recall while scanning only 13% of the corpus and an HNSW-style graph reaching 0.97 recall with $13\times$ fewer distance comparisons than exhaustive search (Listing 4); hybrid search where equal-weight reciprocal-rank fusion is *no free lunch* when one retriever dominates (0.38, between dense 0.26 and BM25 0.42) but *beats both* under vocabulary mismatch when

their errors decorrelate (0.29 versus 0.26 and 0.23) (Listing 5); two-stage reranking lifting recall@10 by 44% and MRR by 66% by rescoring a cheap shortlist with a precise scorer (Listing 6); query transformation via pseudo-relevance feedback, which *degrades* recall here (0.185 $\rightarrow$ 0.164) while an oracle expansion toward the true answer centroid would lift it to 0.41 — signal quality is everything (Listing 7); and a RAGAS-style evaluation where four metrics each catch a *different* failure, context recall collapsing to 0.25 on a retrieval miss, faithfulness to 0.43 on a hallucination, and answer relevance to 0.39 on an evasive answer, while the others stay high (Listing 8).

Why RAG: parametric vs. non-parametric knowledge

The clean way to frame RAG in an interview is as a choice about *where knowledge lives*. **Fine-tuning** writes knowledge into the weights — durable, fast at inference (no retrieval step), and able to change the model's *behavior* and *style*, but expensive to update (every new fact needs a training run), prone to catastrophic forgetting, impossible to attribute to a source, and a poor fit for facts that change daily or that must be access-controlled per user. **Long-context prompting** — just paste the documents into a very large context window — is the simplest alternative and genuinely competes for small corpora, but it scales badly: attention cost grows with context length, relevant facts get *lost in the middle* (Chapter 22), per-query token cost balloons, and you still need a way to *choose* which documents to paste once the corpus exceeds the window. **RAG** keeps knowledge non-parametric in an external store and retrieves just the relevant slice per query, which makes updates trivial (re-index, no retraining), supports citation and access control naturally, and keeps the prompt small — at the cost of a retrieval system that can fail, and an extra latency hop. These are not mutually exclusive: the production norm is *fine-tune the behavior* (how to follow instructions, when to say "I don't know," how to cite) and *retrieve the facts*, with long-context used to stuff more retrieved chunks in when the budget allows.

The reason retrieval quality dominates the whole system is visible in the very first measurement. A RAG answer is upper-bounded by what the retriever surfaces: if the needed passage is not in the retrieved set, no amount of generation skill recovers it — the model either abstains or hallucinates. Listing 1 makes this concrete with a recall/rank curve on the LSA retriever: recall is only 0.19 at rank 10 and 0.37 at rank 50, meaning that even fetching fifty passages leaves most of the gold evidence unretrieved with this weak embedder. That single curve motivates almost everything else in the chapter — better chunking, better similarity, hybrid signals, and reranking all exist to bend that curve upward, and evaluation exists to tell you where on it you actually are.

Chunking: the unit of retrieval

Documents must be split into passages before indexing, and the chunk is the atom the retriever scores and the generator reads, so chunking silently sets the ceiling on

everything downstream. The trade-off is a tug-of-war between *precision of the embedding* and *completeness of the context*. A small chunk yields a focused embedding (one topic, little dilution) and pinpoints the relevant span, but may be too small to actually answer the question and multiplies index size and near-duplicate competition; a large chunk carries enough context to answer but averages several topics into one vector, blurring retrieval, and wastes context-window budget on irrelevant text. The common strategies form a ladder: **fixed-size** (split every N tokens — trivial but slices sentences and ideas mid-thought); **recursive** (split on a priority list of separators — paragraphs, then sentences, then words — so chunks respect natural boundaries, the LangChain default); **semantic** (start a new chunk when consecutive sentences' embeddings diverge past a threshold, so each chunk is topically coherent); and **parent-document / small-to-big** (embed and retrieve *small* chunks for precision but feed the generator the *parent* passage or a window around the hit for context — decoupling the retrieval unit from the generation unit, which is the single most effective practical trick).

Listing 2 measures two of these effects on real documents. First, *query type decides the optimal size*. When the query shares surface terms with the target — a lexical match — retrieval succeeds at essentially every chunk size (recall ~ 0.97 from 40 to 320 words), so you should prefer *smaller* chunks for their lower cost and tighter grounding. But when the query must match by *meaning* to a passage whose exact words were held out — a semantic match — recall climbs steadily with chunk size ($0.09 \rightarrow 0.29 \rightarrow 0.51 \rightarrow 0.65$), because a larger chunk aggregates more weak topical signal for the compressed embedding to lock onto. There is no universal best size; there is a best size *for your query and embedder*. Second, *boundaries matter*: at a fixed 80-word budget, a naive fixed-width cut scores 0.953 while sentence-aware packing scores 0.973 — two points of recall lost purely to fragmenting answer spans across a boundary, which is exactly what recursive and semantic splitting exist to prevent. The practical synthesis is the parent-document pattern: chunk small and sentence-aware for retrieval precision, then hand the generator a larger surrounding window so it has enough to answer.

Embeddings and similarity metrics

Retrieval reduces to geometry: encode the query and every passage as vectors, then rank passages by closeness. The encoder is an **embedding model** — classically LSA/TF-IDF (used here so the mechanics are transparent), in production a trained bi-encoder (Sentence-BERT, E5, GTE, OpenAI/Cohere embeddings) that maps text to a few-hundred-to-few-thousand-dimensional space where semantic similarity is geometric proximity. The three distance choices an interview expects you to compare are **Euclidean** (L_2), **dot product**, and **cosine**, and the relationship among them is the whole point. For vectors of equal norm they induce the *same* ranking; they diverge only when norms vary. Cosine measures the *angle* alone — pure semantic direction, magnitude-invariant. The dot product is $q \cdot p = |q| |p| \cos \theta$, so it multiplies the angle by the passage norm, which means a longer or more "generic" passage with a large norm can outrank a better-aligned but shorter one purely on magnitude. Euclidean distance on *normalized* vectors is

monotonic with cosine ($|a - b|^2 = 2 - 2\cos\theta$), so once you L2-normalize, cosine, dot, and Euclidean all agree — which is why the standard recipe is simply *normalize your embeddings and use dot product* (fast) as a stand-in for cosine.

Listing 3 exhibits the failure mode you get if you skip that normalization. Running every query's top-10 under raw dot product versus cosine on un-normalized LSA vectors, the number of times a passage is retrieved correlates **+0.50** with its vector norm under dot product but only **+0.05** under cosine: the raw dot product systematically over-retrieves high-norm passages regardless of whether they are actually on-topic (the top decile of passages by dot-product retrieval frequency have mean norm 0.534 versus a corpus mean of 0.385). This "length bias" is a real and common bug — an unnormalized index quietly biased toward long or boilerplate passages — and the fix is one line: normalize. The corollary interviewers probe is that the *right* metric is the one your embedding model was **trained with**: models trained with a cosine objective must be scored with cosine; using dot product on them reintroduces exactly this bias.

Vector databases and index structures

Once passages are vectors, retrieval is nearest-neighbour search, and at scale the naive approach — compute the query's distance to all N vectors and sort — is too slow: it is $O(N)$ per query and $O(N)$ memory bandwidth, fine for thousands of vectors but not for the billions a production system holds. **Vector databases** (FAISS as a library; Chroma, Pinecone, Weaviate, Qdrant, Milvus as servers; pgvector as a Postgres extension) exist to make this sublinear via an **approximate nearest neighbour (ANN)** index that trades a small, tunable amount of recall for a large speedup. The two dominant index families are worth being able to sketch from scratch. **IVF** (inverted file) *partitions* the space with k -means into n_{list} cells, stores each vector under its nearest centroid, and at query time scans only the n_{probe} cells nearest the query — turning an N -vector scan into an n_{probe}/n_{list} fraction of one. **HNSW** (hierarchical navigable small world) builds a *proximity graph* — each vector linked to its nearest neighbours — and answers a query by greedily walking the graph toward the query, using a bounded priority queue (ef) of candidates and a hierarchy of coarse-to-fine layers so the walk starts with long hops and ends with short ones; it is the default in most vector databases because it gives the best recall-at-latency, at the cost of higher memory and slow deletes.

Listing 4 builds both and measures the trade-off directly. The from-scratch IVF sweeps n_{probe} from 1 to 64 over 64 cells: recall versus exact search climbs $0.56 \rightarrow 0.72 \rightarrow 0.84 \rightarrow 0.91 \rightarrow 0.96 \rightarrow 0.99 \rightarrow 1.00$, and the companion curve shows *why you tune it* — at $n_{probe}=8$ the index reaches 0.91 recall while scanning only **13%** of the corpus, the characteristic "most of the recall for a fraction of the work" knee that defines ANN. The from-scratch HNSW layer — a k -nearest-neighbour graph searched with an $ef=64$ priority-queue beam — reaches **0.97** recall using an average of 457 distance evaluations versus the 6,020 an exhaustive scan performs, a **13×** reduction in comparisons. (The same graph searched with a naive single-path greedy walk instead of the beam collapses

to 0.27 recall — a compact demonstration of why HNSW's bounded-queue `ef` parameter, not just the graph, is what makes it work, and the knob you raise to trade latency for recall.) The interview headline is the shape of both curves: ANN recall is a *tunable* quantity, `nprobe` and `ef` are the dials, and the whole game is finding the point on the recall/latency curve your product can tolerate.

Hybrid search and reranking

Dense embeddings and classical lexical search fail in *different* directions, which is the entire argument for combining them. A dense bi-encoder matches by meaning, so it handles synonyms and paraphrase but blurs rare, precise tokens — an error code, a product SKU, a surname — into their nearest semantic neighbourhood. **BM25**, the standard sparse lexical scorer (a TF-IDF refinement with term-frequency saturation via k_1 and length normalization via b), does the opposite: it nails exact and rare-term matches but is blind to a synonym it has never seen co-occur. **Hybrid search** runs both and fuses their rankings, most robustly with **Reciprocal Rank Fusion** (RRF), which scores each document by $\sum_i 1/(k + \text{rank}_i)$ across retrievers — using *ranks*, not scores, so it needs no fragile calibration between a cosine and a BM25 magnitude.

Listing 5 implements BM25 from scratch and fuses it with the dense retriever via RRF, and the honest result is a two-part lesson interviewers love. In the **exact-term regime**, where the query carries the document's rare keywords, BM25 (0.42 recall@20) strongly outperforms the weak dense retriever (0.26), and equal-weight RRF lands *between* them at 0.38 — fusion is **not a free lunch**; naively fusing a strong retriever with a much weaker one can drag the strong one down. But in the **vocabulary-mismatch regime** — the realistic case dense retrieval was invented for, simulated by deleting each query's most discriminative terms — BM25 collapses to 0.23 (below dense's 0.26), their errors become independent, and hybrid now **beats both** at 0.29. The precondition for fusion to help is exactly this: the retrievers must be *comparably strong* and their errors *decorrelated*. That is also why you would weight or tune the fusion in practice rather than trust an equal split.

The second stage that consistently helps is **reranking**. First-stage retrieval optimizes *recall* cheaply over the whole corpus; a **cross-encoder reranker** then optimizes *precision* expensively over just the top- N shortlist. The distinction is architectural: a bi-encoder embeds query and passage *separately* (so passage vectors can be pre-computed and indexed), whereas a cross-encoder feeds the query and passage *together* through the model, letting every query token attend to every passage token — far more accurate, but far too slow to run against millions of passages, so it is reserved for reranking a shortlist of tens. Listing 6 stands in a precise reranker (the full high-dimensional TF-IDF similarity the SVD compression discarded) for the cross-encoder and rescores each query's top-50 dense candidates: recall@10 rises 44% (0.202 → 0.292), MRR@50 rises 66% (0.268 → 0.445), and nDCG@10 rises 65% (0.172 → 0.285) — large precision gains for the cost of

scoring fifty passages, the reason essentially every production RAG stack ends in a reranker (Cohere Rerank, bge-reranker, cross-encoder/ms-marco).

Query transformation and contextual compression

The query the user types is often a poor *retrieval* probe: too short, phrased differently from the corpus, or bundling several questions at once. **Query transformation** rewrites it before or during retrieval. **Multi-query** generates several paraphrases with an LLM, retrieves for each, and unions the results — insurance against any single phrasing missing the target. **HyDE** (hypothetical document embeddings) asks the LLM to *hallucinate an answer* to the question and retrieves with *that*, on the logic that a fake answer sits closer in embedding space to the real answer passages than the question does. **Query decomposition** splits a multi-hop question ("compare X's policy to Y's") into sub-questions retrieved independently, the retrieval analogue of chain-of-thought. **Step-back prompting** abstracts a specific question to a more general one to retrieve background. Their classical, LLM-free ancestor is **pseudo-relevance feedback** (PRF / Rocchio): retrieve once, *assume* the top- k are relevant, and move the query vector toward their centroid, $q' = q + \beta \text{mean}(\text{top-}k)$.

Listing 7 measures PRF and reports a deliberately *negative* result, because the precondition it violates is the whole lesson. Sweeping β from 0 to 1, recall@10 *falls* monotonically ($0.185 \rightarrow 0.172 \rightarrow 0.169 \rightarrow 0.168 \rightarrow 0.164$): feedback makes it **worse**. The reason is that the weak first-stage top- k is polluted with off-topic passages, so the "relevant" centroid the query is dragged toward is itself noisy — garbage in, garbage out. The control proves the mechanism is sound when the signal is clean: expanding toward the *true* gold centroid (oracle feedback) lifts recall to **0.41**. The interview-grade takeaway is that every query-expansion method — PRF, HyDE, multi-query — is only as good as the quality of the expansion signal; on a strong retriever they help, on a weak one they amplify noise, and you should measure rather than assume.

A complementary post-retrieval step is **contextual compression**: once passages are retrieved, filter or shrink them before they reach the generator. This spans dropping retrieved chunks whose relevance score is below a threshold, extracting only the sentences within a chunk that pertain to the query (an LLM or a smaller extractive model does this), and re-ordering so the most relevant context sits at the start or end of the prompt where the model attends most (Chapter 22's lost-in-the-middle). The payoff is lower token cost and less distraction for the generator, which measurably improves faithfulness when the retrieved set is noisy.

Advanced RAG architectures

The naive "retrieve-then-read" pipeline has well-known failure modes, and the advanced patterns each target one. **Parent-document (small-to-big) retrieval**, already introduced, decouples the retrieval unit from the generation unit — embed small for

precision, return large for context — and is the highest-value upgrade for the least complexity. **Self-RAG** trains the model to emit *reflection tokens* that decide, per step, *whether* retrieval is even needed (many tokens need no external fact), then to *critique* the retrieved passages for relevance and its own output for support, so retrieval becomes adaptive rather than unconditional. **Corrective RAG (CRAG)** adds a lightweight retrieval *evaluator*: if the retrieved context is judged weak or ambiguous, it triggers a corrective action — a web search fallback, a query rewrite, or knowledge refinement — instead of feeding the generator bad context. **GraphRAG** builds a knowledge graph of entities and relations from the corpus (and community summaries over it), so that *global* questions — "what are the main themes across these documents?" — which flat top-*k* retrieval answers poorly because no single chunk contains the answer, are served by traversing or summarizing the graph. These share a direction: make retrieval *conditional, evaluated, and structured* rather than a fixed one-shot lookup — which is also the seam where RAG blends into **agentic retrieval** (Chapter 25), where a loop decides what to retrieve next based on what it has found so far.

Evaluating RAG

RAG has two components that fail independently — the *retriever* and the *generator* — so a single end-to-end score cannot tell you *what* broke, and evaluation must be componentized. The **RAGAS** framework is the common vocabulary, and its metrics split cleanly along that seam. On the retrieval side: **context recall** (did retrieval fetch all the passages needed to answer — the ceiling on everything) and **context precision** (are the relevant passages ranked *above* the irrelevant ones in what was retrieved). On the generation side: **faithfulness** (is every claim in the answer *entailed by* the retrieved context, i.e., grounded rather than hallucinated) and **answer relevance** (does the answer actually address the question, rather than being correct-but-evasive or padded). In practice these are computed with an LLM-as-judge (Chapter 23) decomposing answers into claims and checking entailment — which inherits the judge biases from that chapter and must itself be validated against human labels.

Listing 8 is an exact simulation that implements each metric from its definition and runs four controlled scenarios to demonstrate the core reason there are four metrics: **each catches a failure the others miss**. A *good* pipeline scores high everywhere (all four ≥ 0.93). A *retrieval miss* (the gold passages simply were not fetched) is caught by context recall crashing to 0.25 and drags faithfulness down to 0.54 because the answer cannot be grounded — yet answer relevance stays at 0.99, since a fluent, on-topic, *wrong* answer looks fine to a relevance check. A *hallucination* (good retrieval, but the answer asserts things the context never supports) is invisible to the context metrics (both stay ≥ 0.95) and is caught *only* by faithfulness collapsing to 0.43. And an *evasive answer* (well-grounded but dodging the actual question) is caught *only* by answer relevance falling to 0.39 while everything else stays high. This orthogonality is the interview point: you cannot compress RAG quality to one number, because "the retrieval was wrong," "the

model made something up," and "the model didn't answer" are different bugs with different fixes, and only a *panel* of metrics localizes which one you have.

Code implementations

(Listing 1 defines the shared retrieval machinery — corpus construction, LSA embedding, and the recall evaluator — as `rag_common.py`, reused by the rest via `from rag_common import ...`. All listings run on the real 20 Newsgroups corpus with classical LSA embeddings so every retrieval mechanic is visible and reproducible; the numbers in the prose are this code's actual output.)

Listing 1 — Retrieval recall@k — the ceiling on RAG

The shared machinery plus the first measurement: dense retrieval recall as a function of how many passages you fetch. The curve rises slowly and saturates low with a weak embedder — every later technique exists to lift it.

```

"""Listing 1: shared RAG retrieval machinery + recall@k demo.
Builds a passage corpus from 20newsgroups, embeds it with LSA (TF-IDF -> truncated
SVD),
and defines a self-supervised retrieval task: each doc's FIRST passage is a query whose
GOLD
relevant set is the OTHER passages of the same document. All later listings import
this."""
import os, re, numpy as np
os.environ.setdefault("OMP_NUM_THREADS", "4")
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD

CATS = ['comp.graphics', 'rec.sport.hockey', 'sci.space', 'sci.med']
_WORD = re.compile(r"[A-Za-z]{2,}")

def _passages(text, size=60):
    """Split a document into ~size-word passages on whitespace (paragraph/sentence
agnostic)."""
    toks = text.split()
    return [" ".join(toks[i:i+size]) for i in range(0, len(toks), size)]

def build_corpus(min_doc_words=140, psize=60, seed=0):
    raw = fetch_20newsgroups(subset='train', categories=CATS,
                             remove=('headers', 'footers', 'quotes'), random_state=seed)
    passages, doc_of, first_of_doc, cat_of = [], [], {}, []
    for di, (txt, lab) in enumerate(zip(raw.data, raw.target)):
        if len(txt.split()) < min_doc_words: continue
        ps = _passages(txt, psize)
        ps = [p for p in ps if len(p.split()) >= 15]
        if len(ps) < 2: continue
        first_of_doc[di] = len(ps) # index of this doc's first passage
        (the query)
        for p in ps:
            passages.append(p); doc_of.append(di); cat_of.append(lab)
    doc_of = np.array(doc_of); cat_of = np.array(cat_of)
    queries = np.array(sorted(first_of_doc.values())) # passage-idx used as
    query
    return dict(passages=passages, doc_of=doc_of, cat_of=cat_of, queries=queries)

def embed_lsa(passages, dim=128, seed=0):

```

```

tf = TfidfVectorizer(min_df=3, max_df=0.5, sublinear_tf=True, stop_words='english')
X = tf.fit_transform(passages) # sparse TF-IDF (for
BM25/rerank too)
svd = TruncatedSVD(n_components=dim, random_state=seed)
Z = svd.fit_transform(X) # dense LSA
Zn = Z / (np.linalg.norm(Z,axis=1,keepdims=True)+1e-9) # unit vectors -> cosine
= dot
    return dict(tfidf=tf, X=X, Z=Z, Zn=Zn)

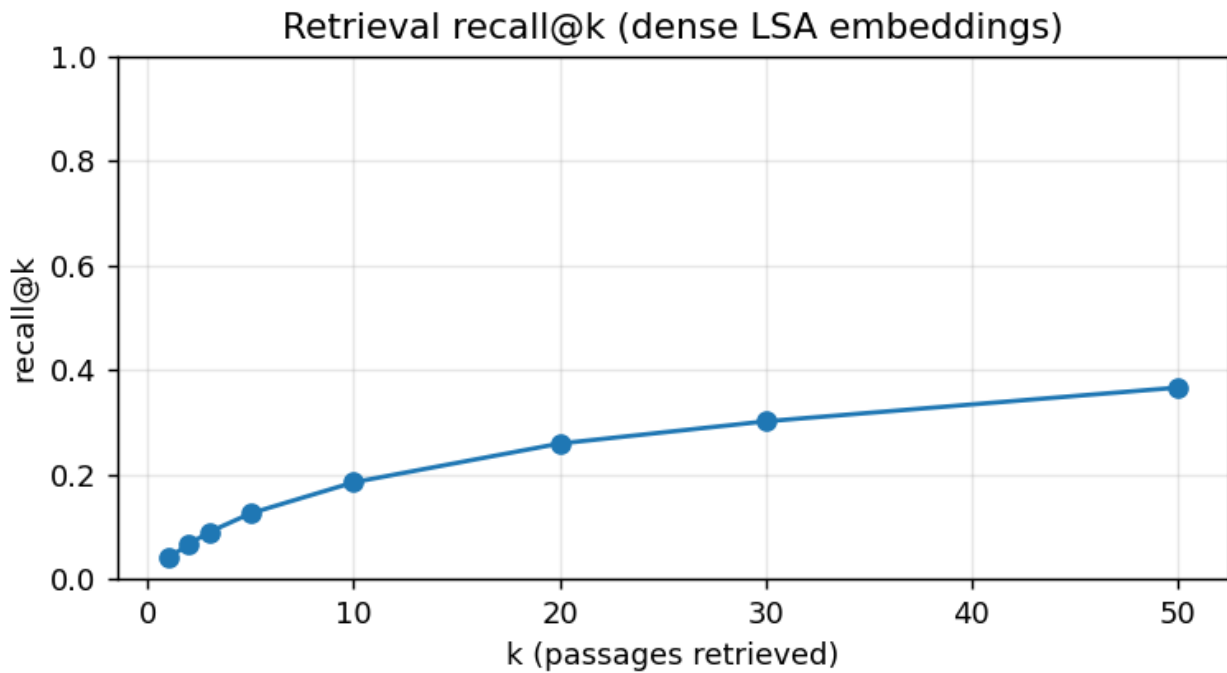
def gold_sets(C):
    """gold[q] = set of passage indices in the same doc as q, excluding q itself."""
    doc_of, queries = C['doc_of'], C['queries']
    by_doc = {}
    for i,d in enumerate(doc_of): by_doc.setdefault(d,[]).append(i)
    gold = {}
    for q in queries:
        d = doc_of[q]; gold[q] = set(by_doc[d]) - {q}
    return gold

def recall_at_k(scores, q, gold_q, ks, exclude):
    order = np.argsort(-scores)
    order = order[order != exclude] # drop the query passage
    itself
    g = gold_q
    if not g: return None
    out = {}
    for k in ks:
        hit = sum(1 for i in order[:k] if i in g)
        out[k] = hit/len(g)
    return out

def dense_scores(Zn, qvec): # cosine (unit vectors)
    return Zn @ qvec

if __name__ == "__main__":
    import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt, time
    FIG="figures/ch24"; os.makedirs(FIG, exist_ok=True)
    t0=time.time()
    C = build_corpus(); E = embed_lsa(C['passages']); Zn=E['Zn']
    gold = gold_sets(C); ks=[1,2,3,5,10,20,30,50]
    print(f"passages={len(C['passages'])} queries={len(C['queries'])}")
    dim={Zn.shape[1]} build {time.time()-t0:.1f}s")
    agg={k:[] for k in ks}
    for q in C['queries']:
        r = recall_at_k(dense_scores(Zn, Zn[q]), q, gold[q], ks, exclude=q)
        if r:
            for k in ks: agg[k].append(r[k])
    rec={k:float(np.mean(agg[k])) for k in ks}
    for k in ks: print(f" recall@{k:<3d} = {rec[k]:.3f}")
    plt.figure(figsize=(6,3.4))
    plt.plot(ks,[rec[k] for k in ks],"o-",color="#1f77b4")
    plt.xlabel("k (passages retrieved)"); plt.ylabel("recall@k"); plt.ylim(0,1.0)
    plt.title("Retrieval recall@k (dense LSA embeddings)"); plt.grid(alpha=.3);
    plt.tight_layout()
    plt.savefig(f"{FIG}/l1_recall.png",dpi=130); print("saved l1_recall.png")

```



Listing 2 — Chunk size, query type, and boundary awareness

Lexical queries are found at any chunk size; semantic queries need larger chunks to aggregate weak signal; and a naive fixed-width cut loses recall to boundary damage versus sentence-aware packing.

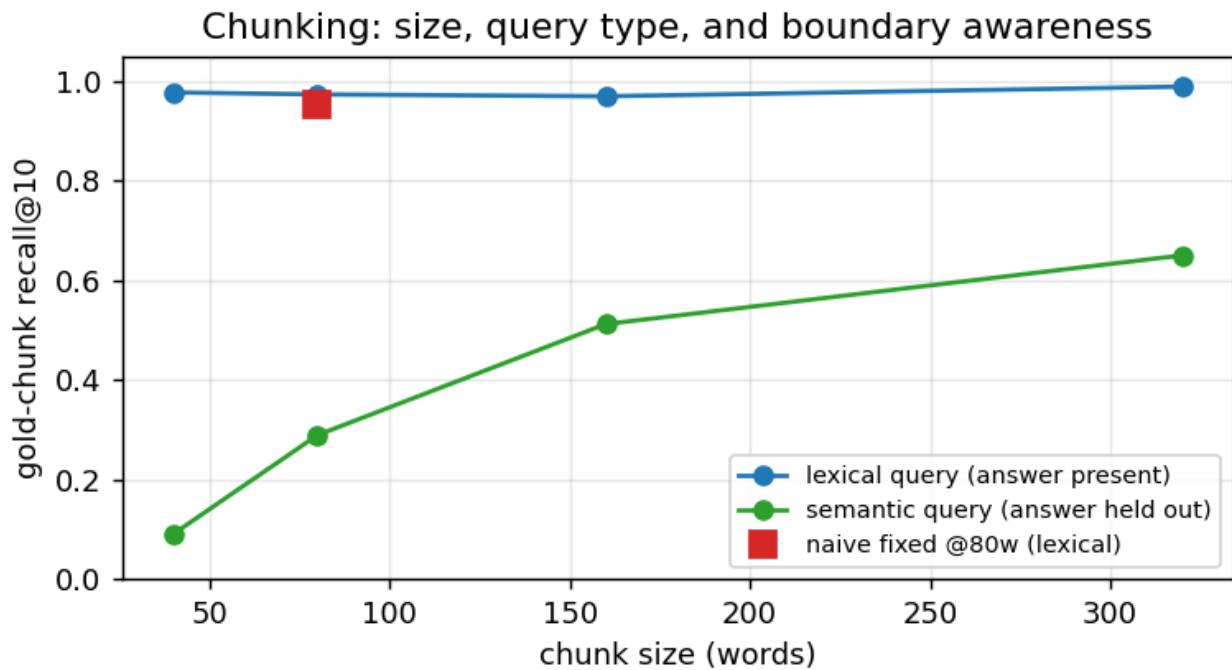
"""Listing 2: chunk size, query type, and boundary awareness. Two retrieval regimes on the same docs: (a) LEXICAL -- the query is a sentence that stays in the corpus (answer text present), gold = its home chunk; (b) SEMANTIC -- the query sentence is HELD OUT and we must retrieve its neighborhood chunk from surrounding context. Lexical retrieval succeeds at any size (so smaller = cheaper, more localized); semantic retrieval IMPROVES with size because a bigger chunk aggregates more weak signal. A naive fixed-width cut fragments answer spans -> boundary damage vs sentence-aware. Metric: recall@10 of the gold chunk."""

```
import os, re, numpy as np, matplotlib; matplotlib.use("Agg"); import
matplotlib.pyplot as plt, time
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
FIG="figures/ch24"; os.makedirs(FIG,exist_ok=True)
CATS=['comp.graphics','rec.sport.hockey','sci.space','sci.med']
raw=fetch_20newsgroups(subset='train',categories=CATS,remove=('headers','footers','quotes'),random_s
SENT=re.compile(r"(?<=[.!?])\s+"); SEN="zqxjneedle"
docs=[]
for t in raw.data:
    ss=[s for s in SENT.split(t) if 12<=len(s.split())<=40]
    if len(t.split())>=220 and len(ss)>=2: docs.append((t, ss[len(ss)//2]))
print(f"docs usable: {len(docs)}")
def ck_fixed(x,s): T=x.split(); return [" ".join(T[i:i+s]) for i in range(0,len(T),s)]
def ck_sent(x,s):
    out,b,n=[],[],0
    for w in (q.split() for q in SENT.split(x)):
```

```

        if not w: continue
        if n+len(w)>s and b: out.append(" ".join(b)); b,n=[],0
        b+=w; n+=len(w)
    if b: out.append(" ".join(b))
    return out
def run(size,mode,hold_out):
    ck=ck_sent if mode=="sent" else ck_fixed
    passages,home,queries=[],[],[]
    for txt,needle in docs:
        src=txt.replace(needle," "+SEN+" ",1) if hold_out else txt
        chunks=ck(src,size)
        gi=[i for i,c in enumerate(chunks) if (SEN in c) or (not hold_out and needle[:
25] in c)]
        if not gi: continue
        g=gi[0]; chunks=[re.sub(r"\s+", " ",c.replace(SEN," ")).strip() for c in chunks]
        if len(chunks[g].split())<5: continue
        base=len(passages); passages+=chunks; home.append(base+g);
    queries.append(needle)
    tf=TfidfVectorizer(min_df=3,max_df=0.5,sublinear_tf=True,stop_words='english')
    svd=TruncatedSVD(n_components=128,random_state=0)
    Zp=svd.fit_transform(tf.fit_transform(passages)); Zp/
    =(np.linalg.norm(Zp,axis=1,keepdims=True)+1e-9)
    Zq=svd.transform(tf.transform(queries)); Zq/
    =(np.linalg.norm(Zq,axis=1,keepdims=True)+1e-9)
    S=Zq@Zp.T; return sum(1 for r,g in enumerate(home) if g in np.argsort(-S[r][:10])/
len(home))
sizes=[40,80,160,320]; t0=time.time()
lex=[run(s,"sent",False) for s in sizes]
sem=[run(s,"sent",True) for s in sizes]
lex_naive=run(80,"fixed",False)
for i,s in enumerate(sizes): print(f"size={s:3d}: lexical R@10={lex[i]:.3f} semantic
R@10={sem[i]:.3f}")
print(f"naive-fixed @80 lexical R@10={lex_naive:.3f} (sentence-aware @80 = {lex[1]:.
3f})")
print(f"elapsed {time.time()-t0:.1f}s")
plt.figure(figsize=(6,3.4))
plt.plot(sizes,lex,"o-",color="#1f77b4",label="lexical query (answer present)")
plt.plot(sizes,sem,"o-",color="#2ca02c",label="semantic query (answer held out)")
plt.plot([80],[lex_naive],"s",color="#d62728",ms=9,label="naive fixed @80w (lexical)")
plt.xlabel("chunk size (words)"); plt.ylabel("gold-chunk recall@10"); plt.ylim(0,1.05)
plt.title("Chunking: size, query type, and boundary awareness"); plt.legend(fontsize=8)
plt.grid(alpha=.3); plt.tight_layout(); plt.savefig(f"{FIG}/l2_chunking.png",dpi=130);
print("saved l2_chunking.png")

```



Listing 3 — Why cosine, not raw dot product

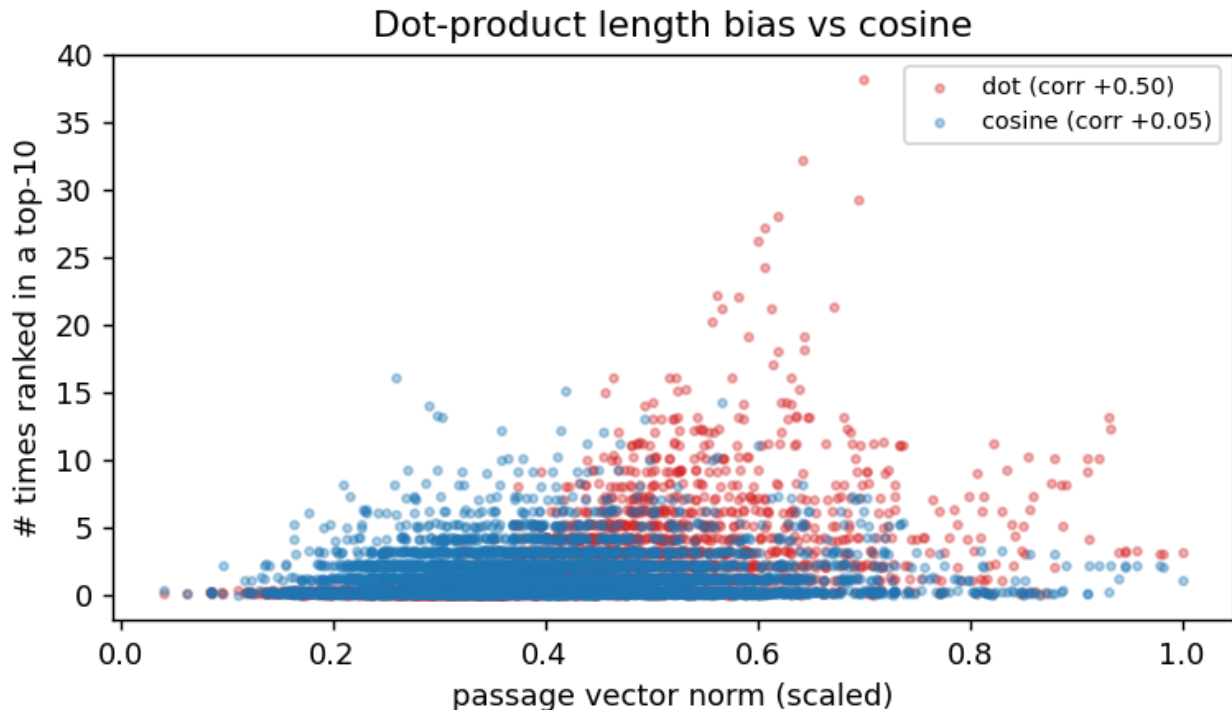
With un-normalized embeddings the dot product inflates high-norm passages into rankings they do not belong in — a length bias that correlates +0.50 with norm versus +0.05 for cosine.

```

"""Listing 3: why cosine, not raw dot product. With UN-normalized embeddings the dot
product
 $q \cdot p = |q||p|\cos(\theta)$  is inflated by the passage norm  $|p|$ , so high-norm passages get
retrieved for
queries they are not actually about (a popularity/length bias). Cosine divides  $|p|$  out.
We rank every
query's top-10 under each metric, tally how often each passage appears, and correlate
that tally with
the passage's vector norm: dot correlates strongly (bias), cosine weakly."""
import os, numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt, time
from rag_common import build_corpus, embed_lsa
FIG="figures/ch24"; os.makedirs(FIG,exist_ok=True)
t0=time.time(); C=build_corpus(); E=embed_lsa(C['passages'])
Z=E['Z']; Zn=E['Zn']; q=C['queries']; P=Z.shape[0]
norm=np.linalg.norm(Z,axis=1)
cnt_dot=np.zeros(P); cnt_cos=np.zeros(P)
for qi in q:
    dot=Z@Z[qi]; cos=Zn@Zn[qi]
    for arr,cnt in ((dot,cnt_dot),(cos,cnt_cos)):
        order=np.argsort(-arr); order=order[order!=qi][:10]
        cnt[order]+=1
cd=np.corrcoef(norm,cnt_dot)[0,1]; cc=np.corrcoef(norm,cnt_cos)[0,1]
print(f"passages={P} build {time.time()-t0:.1f}s")
print(f"corr(norm, #top10 appearances): dot={cd:.2f} cosine={cc:.2f}")
print(f"mean norm of top-decile-by-dot-frequency passages vs overall: "
      f"{norm[np.argsort(-cnt_dot)[:P//10]].mean():.3f} vs {norm.mean():.3f}")
nn=norm/norm.max()
plt.figure(figsize=(6,3.6))
plt.scatter(nn,cnt_dot+np.random.default_rng(0).uniform(0,.3,P),s=7,alpha=.
35,color="#d62728",label=f"dot (corr +{cd:.2f})")

```

```
plt.scatter(nn,cnt_cos+np.random.default_rng(1).uniform(0,.3,P),s=7,alpha=.
35,color="#1f77b4",label=f"cosine (corr +{cc:.2f})")
plt.xlabel("passage vector norm (scaled)"); plt.ylabel("# times ranked in a top-10")
plt.title("Dot-product length bias vs cosine"); plt.legend(fontsize=8);
plt.tight_layout()
plt.savefig(f"{FIG}/l3_similarity.png",dpi=130); print("saved l3_similarity.png")
```



Listing 4 — ANN indexes from scratch: IVF and HNSW

IVF partitions with k-means and probes the nearest cells; HNSW walks a proximity graph with a bounded priority queue. Both reach high recall while doing a fraction of exhaustive search's work.

```
"""Listing 4: approximate nearest-neighbour indexes from scratch. Exhaustive (flat)
search is exact
but O(N) per query. Two production index families trade a little recall for large
speedups:
IVF partitions vectors into cells with k-means and only scans the nprobe nearest cells;
HNSW/NSW
builds a proximity graph and greedily walks toward the query. We build both over the
LSA passages and
measure recall vs the exact top-10 as a function of work done."""
import os, numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt, time
from rag_common import build_corpus, embed_lsa
FIG="figures/ch24"; os.makedirs(FIG,exist_ok=True)
rng=np.random.default_rng(0); t0=time.time()
C=build_corpus(); Zn=embed_lsa(C['passages'])['Zn']; q=C['queries']; N=Zn.shape[0]
K=10
exact={qi:set(np.argsort(-(Zn@Zn[qi]))[:K+1]) - {qi} for qi in q} # gold top-10
(drop self)
def recall(cand,qi):
    g=set(list(exact[qi])[:K]); return len(set(cand)&g)/K

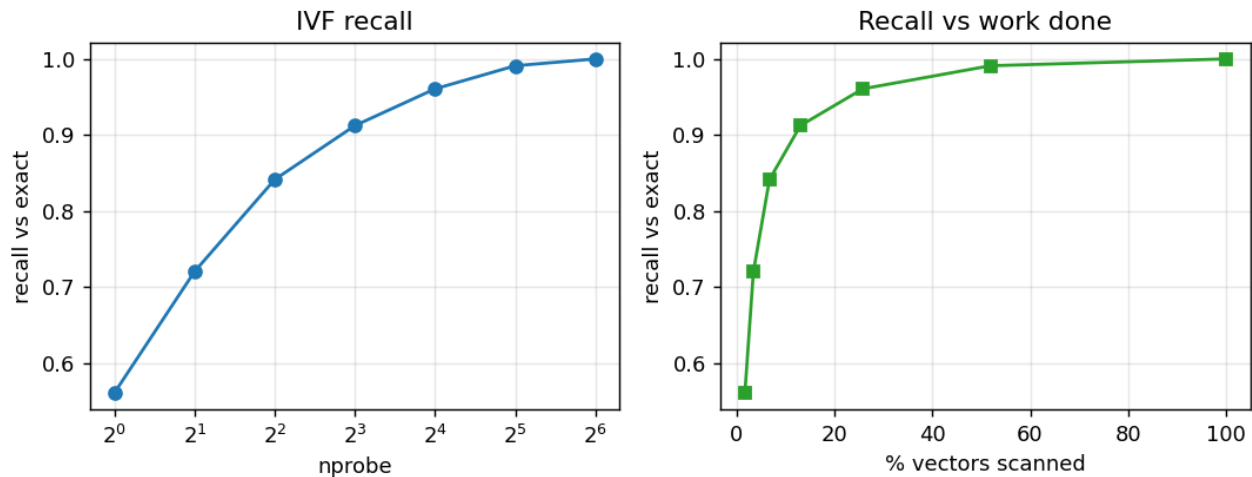
# ---- IVF: k-means cells (from-scratch Lloyd), probe nprobe nearest centroids ----
```

```

nlist=64
cen=Zn[rng.choice(N,nlist,replace=False)].copy()
for _ in range(12):
    asg=np.argmax(Zn@cen.T,axis=1) # cosine assignment
    for c in range(nlist):
        m=Zn[asg==c]
        if len(m): cen[c]=m.mean(0); cen[c]/=np.linalg.norm(cen[c])+1e-9
cells={c:np.where(asg==c)[0] for c in range(nlist)}
def ivf_search(qi,nprobe):
    probes=np.argsort(-(cen@Zn[qi]))[:nprobe]
    cand=np.concatenate([cells[c] for c in probes]) if nprobe else np.array([],int)
    scanned=len(cand)
    if scanned==0: return [],0
    sub=cand[np.argsort(-(Zn[cand]@Zn[qi]))]
    sub=sub[sub!=qi][:K]; return sub,scanned
nprobes=[1,2,4,8,16,32,64]; rec=[]; frac=[]
for npb in nprobes:
    rs=[]; sc=[]
    for qi in q:
        c,s=ivf_search(qi,npb); rs.append(recall(c,qi)); sc.append(s/N)
    rec.append(np.mean(rs)); frac.append(np.mean(sc)*100)
    print(f"IVF nprobe={npb:2d}: recall@10={rec[-1]:.3f} scanned={frac[-1]:.1f}% of
index")

# ---- NSW proximity graph (HNSW's core layer): kNN graph + greedy best-first walk ----
M=16
S=Zn@Zn.T; np.fill_diagonal(S,-1)
nbr=np.argsort(-S,axis=1)[:,:M] # M nearest neighbours
per node
import heapq
def beam_search(qi,ef=64):
    """HNSW layer-0 search: min-heap of candidates, bounded result set of size ef."""
    entry=int(rng.integers(N)); d0=1-(Zn[entry]@Zn[qi])
    visited={entry}; cand=[(d0,entry)]; res=[(-d0,entry)]; evals=1 # res is a max-
heap by distance
    while cand:
        dc,c=heapq.heappop(cand)
        if -res[0][0] < dc and len(res)>=ef: break
# nearest candidate worse than worst kept
        for nb in nbr[c]:
            if nb in visited: continue
            visited.add(nb); evals+=1; d=1-(Zn[nb]@Zn[qi])
            if len(res)<ef or d < -res[0][0]:
                heapq.heappush(cand,(d,nb)); heapq.heappush(res,(-d,nb))
            if len(res)>ef: heapq.heappop(res)
        cand_ids=[i for _,i in sorted(res,key=lambda x:x[0]) if i!=qi][::-1][:K]
        return cand_ids,evals
nrec=[]; nev=[]
for qi in q:
    c,e=beam_search(qi); nrec.append(recall(c,qi)); nev.append(e)
print(f"HNSW layer M={M}: recall@10={np.mean(nrec):.3f} avg dist-vals={np.mean(nev):.
0f} "
    f"(flat scans {N}) -> {N/np.mean(nev):.0f}x fewer comparisons")
print(f"elapsed {time.time()-t0:.1f}s")
fig,ax=plt.subplots(1,2,figsize=(8.4,3.4))
ax[0].plot(range(len(nprobes)),rec,"o-",color="#1f77b4")
ax[0].set_xticks(range(len(nprobes))); ax[0].set_xticklabels([f"$2^{{{i}}}$" for i in
range(len(nprobes))])
ax[0].set_xlabel("nprobe"); ax[0].set_ylabel("recall vs exact"); ax[0].set_title("IVF
recall"); ax[0].grid(alpha=.3)
ax[1].plot(frac,rec,"s-",color="#2ca02c"); ax[1].set_xlabel("% vectors scanned");
ax[1].set_ylabel("recall vs exact")
ax[1].set_title("Recall vs work done"); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.savefig(f"{FIG}/l4_index.png",dpi=130); print("saved
l4_index.png")

```



Listing 5 — Hybrid search: dense + BM25 fused with RRF

BM25 from scratch, fused with dense retrieval by reciprocal rank fusion. Fusion is no free lunch when one retriever dominates, but beats both when their errors decorrelate under vocabulary mismatch.

```

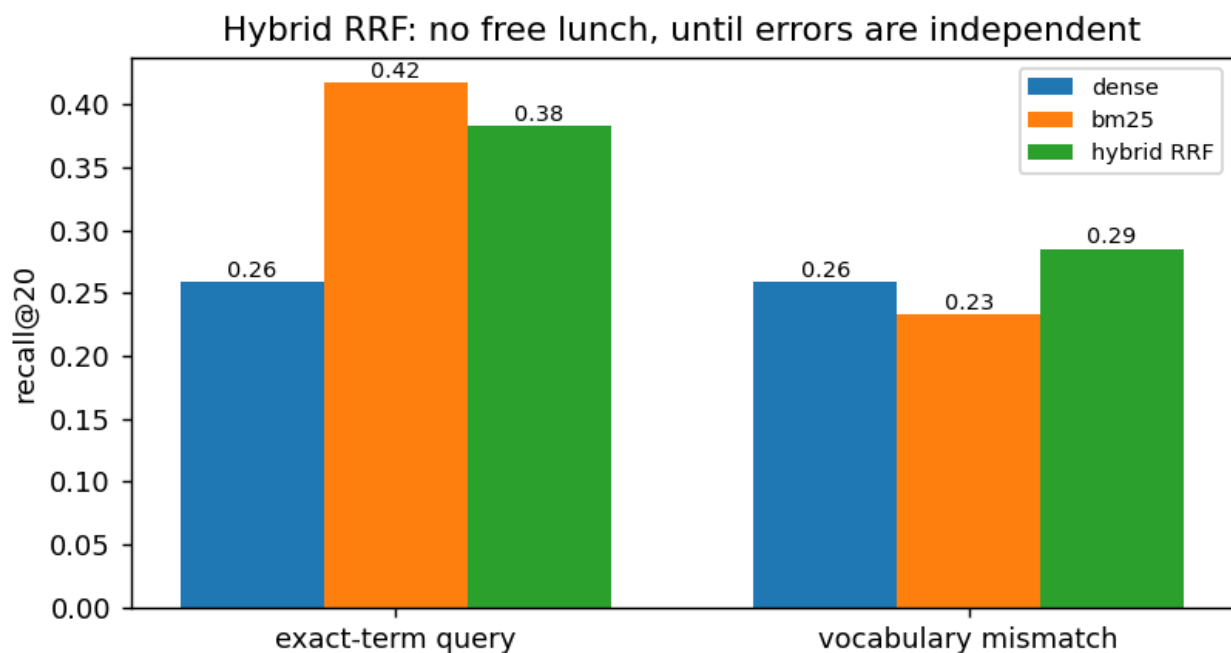
"""Listing 5: hybrid retrieval = dense (cosine LSA) + sparse (BM25) fused with
Reciprocal Rank
Fusion, shown in two regimes. When the query contains the document's exact rare terms,
BM25 dominates
the weak 128-d dense retriever and equal-weight RRF is NO free lunch (it lands between
them). Under
VOCABULARY MISMATCH -- the realistic case dense retrieval exists for, simulated by
deleting each
query's rarest/most-discriminative terms -- BM25 collapses toward dense, their errors
become
independent, and fusion beats both. BM25 built from scratch as a sparse length-
normalized matrix."""
import os, re, math, numpy as np, scipy.sparse as sp, matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt, time
from collections import Counter, defaultdict
from rag_common import build_corpus, embed_lsa, gold_sets
FIG="figures/ch24"; os.makedirs(FIG,exist_ok=True)
t0=time.time(); C=build_corpus(); Zn=embed_lsa(C['passages'])['Zn']
passages=C['passages']; q=C['queries']; gold=gold_sets(C); N=len(passages)
tok=lambda s:[w for w in re.findall(r"[a-z]{2,}",s.lower())]
docs=[tok(p) for p in passages]; dl=np.array([len(d) for d in docs],float);
avgdL=dl.mean()
vocab={}
for d in docs:
    for w in set(d): vocab.setdefault(w,len(vocab))
V=len(vocab); df=np.zeros(V); rows=[]; cols=[]; vals=[]; k1,b=1.5,0.75
for i,d in enumerate(docs):
    c=Counter(d); nz=k1*(1-b+b*dl[i]/avgdL)
    for w,f in c.items():
        j=vocab[w]; df[j]+=1; rows.append(i); cols.append(j); vals.append(f*(k1+1)/
(f+nz))
idf=np.log(1+(N-df+0.5)/(df+0.5))
W=sp.csr_matrix((vals,(rows,cols)),shape=(N,V)).multiply(idf).tocsr()
def qvec(qi,ndrop=0):
    c=[w for w in docs[qi] if w in vocab]
    if ndrop:
        keep=sorted(set(c),key=lambda w:-idf[vocab[w]])[ndrop:]; c=[w for w in c if w
in keep]
    r=np.zeros(V)

```

```

    for w in set(c): r[vocab[w]]=1.0
    return r
DE=Zn[q]@Zn.T
def rrf(*rls,k=60):
    s=defaultdict(float)
    for rl in rls:
        for rank,i in enumerate(rl): s[i]+=1.0/(k+rank+1)
    return [i for i,_ in sorted(s.items(),key=lambda x:-x[1])]
def rk(sc,qi,k=200): o=np.argsort(-sc); return list(o[o!=qi][:k])
K=20
def rec(cand,qi): g=gold[qi]; return len(set(cand[:K])&g)/len(g)
def regime(ndrop):
    BM=(W@np.stack([qvec(qi,ndrop) for qi in q]).T).T
    rd=[];rb=[];rh=[]
    for r,qi in enumerate(q):
        if not gold[qi]: continue
        d=rk(DE[r],qi); b=rk(BM[r],qi); h=[i for i in rrf(d,b) if i!=qi][:K]
        rd.append(rec(d,qi)); rb.append(rec(b,qi)); rh.append(rec(h,qi))
    return np.mean(rd),np.mean(rb),np.mean(rh)
exact=regime(0); mism=regime(12)
print(f"passages={N} vocab={V} build {time.time()-t0:.1f}s")
print(f"EXACT query    : dense={exact[0]:.3f} bm25={exact[1]:.3f} hybrid={exact[2]:.3f}")
print(f"MISMATCH query: dense={mism[0]:.3f} bm25={mism[1]:.3f} hybrid={mism[2]:.3f}")
x=np.arange(2); w=0.25
fig,ax=plt.subplots(figsize=(6.4,3.5))
for k,(lab,col,vals) in enumerate([("dense","#1f77b4",[exact[0],mism[0]]),
                                   ("bm25","#ff7f0e",[exact[1],mism[1]]),
                                   ("hybrid RRF","#2ca02c",[exact[2],mism[2]])]):
    bb=ax.bar(x+(k-1)*w,vals,w,label=lab,color=col)
    for r,v in zip(bb,vals): ax.text(r.get_x()+r.get_width()/2,v+0.004,f"{v:.2f}",ha="center",fontsize=8)
ax.set_xticks(x); ax.set_xticklabels(["exact-term query","vocabulary mismatch"])
ax.set_ylabel(f"recall@{K}");
ax.set_title("Hybrid RRF: no free lunch, until errors are independent")
ax.legend(fontsize=8); plt.tight_layout(); plt.savefig(f"{FIG}/l5_hybrid.png",dpi=130); print("saved l5_hybrid.png")

```



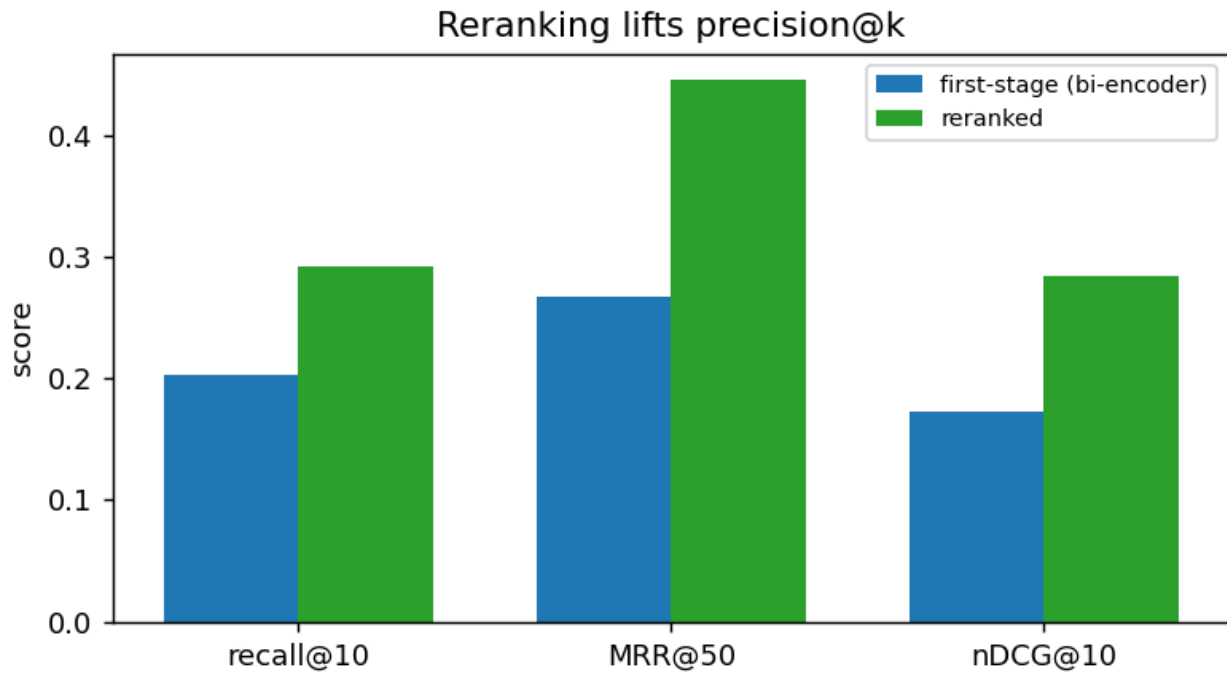
Listing 6 — Two-stage retrieve-and-rerank

A cheap bi-encoder shortlist rescored by a precise query-passage scorer (the full TF-IDF the SVD discarded, standing in for a cross-encoder) lifts recall@10, MRR, and nDCG substantially.

```

"""Listing 6: two-stage retrieve-and-rerank. A cheap bi-encoder (128-d LSA cosine)
encodes query and
passages SEPARATELY and is fast enough to search the whole index, but its lossy
compression blurs
fine distinctions. A reranker rescopes ONLY the top-N shortlist with a more expensive,
more precise
query-passage scorer -- here the full high-dimensional TF-IDF cosine that the SVD threw
away (a stand
-in for a cross-encoder that jointly reads query+passage). Reranking the shortlist
lifts precision@k
metrics without paying the reranker's cost on the whole corpus."""
import os, numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt, time
from sklearn.preprocessing import normalize
from rag_common import build_corpus, embed_lsa, gold_sets
FIG="figures/ch24"; os.makedirs(FIG,exist_ok=True)
t0=time.time(); C=build_corpus(); E=embed_lsa(C['passages'])
Zn=E['Zn']; X=normalize(E['X']); q=C['queries']; gold=gold_sets(C)           # X = full
sparse tfidf, L2-normed
NR=50 # shortlist size
def dcg(rel): return sum(r/np.log2(i+2) for i,r in enumerate(rel))
def ndcg(order,g,k=10):
    rel=[1 if i in g else 0 for i in order[:k]]
    idcg=dcg(sorted([1]*min(len(g),k)+[0]*k,reverse=True)[:k]) or 1
    return dcg(rel)/idcg
def mrr(order,g):
    for i,d in enumerate(order):
        if d in g: return 1/(i+1)
    return 0.0
r10a=r10b=mrra=mrrb=nda=ndb=0; n=0
for r,qi in enumerate(q):
    g=gold[qi]
    if not g: continue
    n+=1
    first=np.argsort(-(Zn@Zn[qi])); first=first[first!=qi][:NR]           # bi-
encoder shortlist
    cross=(X[first]@X[qi].T).toarray().ravel()                         # rerank
scores (full tfidf)
    rer=first[np.argsort(-cross)]                                       # reranked
order
    r10a+=len(set(first[:10])&g)/min(len(g),10); r10b+=len(set(rer[:10])&g)/min(len(g),
10)
    mrra+=mrr(first,g); mrrb+=mrr(rer,g); nda+=ndcg(first,g); ndb+=ndcg(rer,g)
A=[r10a/n,mrra/n,nda/n]; B=[r10b/n,mrrb/n,ndb/n]
print(f"queries={n} shortlist N={NR} build {time.time()-t0:.1f}s")
for lab,a,b in zip(["recall@10","MRR@50","nDCG@10"],A,B):
    print(f" {lab:9s}: first-stage {a:.3f} -> reranked {b:.3f} ({100*(b-a)/a:.0f}
%)" )
x=np.arange(3); w=0.36
fig,ax=plt.subplots(figsize=(6.2,3.5))
ax.bar(x-w/2,A,w,label="first-stage (bi-encoder)",color="#1f77b4")
ax.bar(x+w/2,B,w,label="reranked",color="#2ca02c")
ax.set_xticks(x); ax.set_xticklabels(["recall@10","MRR@50","nDCG@10"]);
ax.set_ylabel("score")
ax.set_title("Reranking lifts precision@k"); ax.legend(fontsize=8)
plt.tight_layout(); plt.savefig(f"{FIG}/l6_rerank.png",dpi=130); print("saved
l6_rerank.png")

```



Listing 7 — Query transformation via pseudo-relevance feedback

PRF (Rocchio) moves the query toward the top-k centroid. Here it degrades recall because the feedback is polluted; an oracle expansion toward the true centroid would help — signal quality is everything.

```

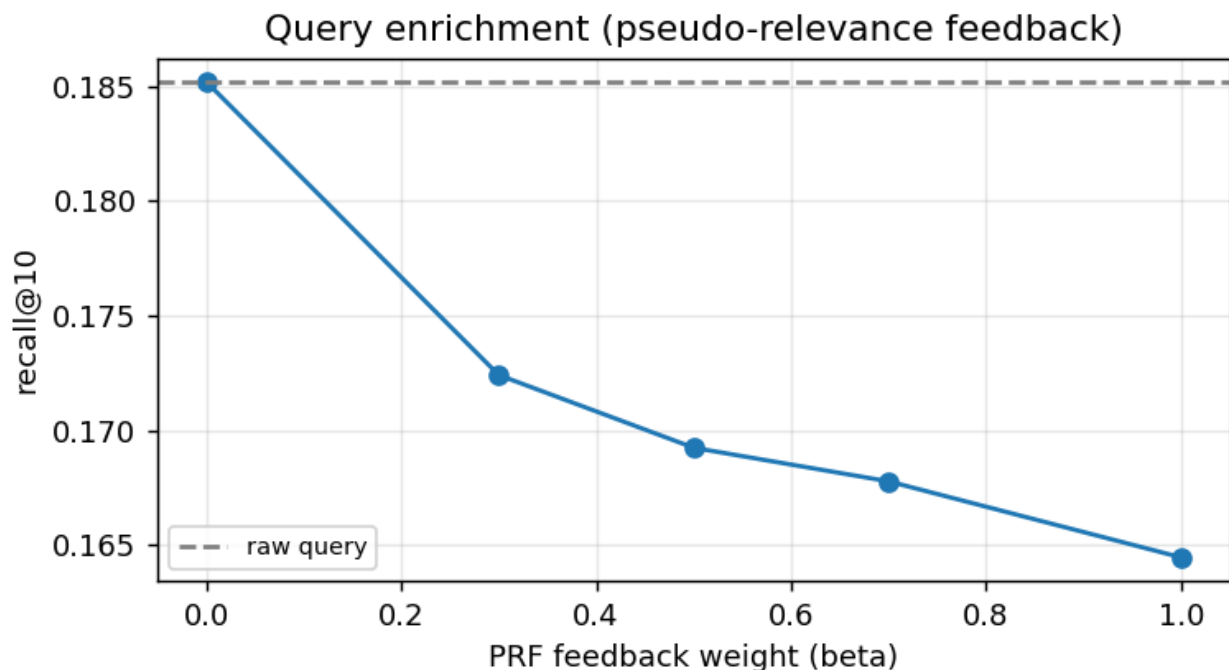
"""Listing 7: query transformation. The query and the answer passages often do not
share surface
form, so rewriting the query can help -- multi-query (ask several rephrasings and union
the hits),
HyDE (embed a hypothetical answer instead of the question), decomposition (split a
multi-hop question).
Their classical, LLM-free ancestor is pseudo-relevance feedback (Rocchio): retrieve
once, ASSUME the
top-k are relevant, and push the query vector toward their centroid: q' = q + beta *
mean(top-k).
This listing measures PRF and reports an HONEST result -- on this corpus feedback
mostly HURTS,
because the weak first-stage top-k is polluted, so the 'relevant' centroid drags the
query off target.
The lesson: query expansion helps only when the feedback signal is cleaner than the
original query."""
import os, numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt, time
from rag_common import build_corpus, embed_lsa, gold_sets
FIG="figures/ch24"; os.makedirs(FIG,exist_ok=True)
t0=time.time(); C=build_corpus(); Zn=embed_lsa(C['passages'])['Zn']; q=C['queries'];
gold=gold_sets(C)
def recall10(vec,qi):
    o=np.argsort(-(Zn@vec)); o=o[o!=qi][:10]; g=gold[qi]
    return len(set(o)&g)/len(g)
def prf(qi,beta,fb=5):
    base=Zn[qi]; o=np.argsort(-(Zn@base)); o=o[o!=qi][:fb]
    cen=Zn[o].mean(0)
    v=base+beta*cen; return v/(np.linalg.norm(v)+1e-9)
betas=[0.0,0.3,0.5,0.7,1.0]; recs=[]
for be in betas:

```

```

rr=[recall10(prf(qi,be),qi) for qi in q if gold[qi]]
recs.append(np.mean(rr)); print(f"PRF beta={be:.1f}: recall@10={recs[-1]:.4f}")
print(f"raw query (beta=0) recall@10={recs[0]:.4f}    build {time.time()-t0:.1f}s")
# oracle feedback: expand toward the TRUE gold centroid -> shows expansion CAN help
with clean signal
orc=[]
for qi in q:
    if not gold[qi]: continue
    cen=Zn[list(gold[qi])].mean(0); v=Zn[qi]+0.5*cen; v/=np.linalg.norm(v)+1e-9
    orc.append(recall10(v,qi))
print(f"ORACLE feedback (toward true gold centroid) recall@10={np.mean(orc):.4f} <-
signal quality is everything")
plt.figure(figsize=(6,3.4))
plt.plot(betas,recs,"o-",color="#1f77b4")
plt.axhline(recs[0],ls="--",color="gray",label="raw query")
plt.xlabel("PRF feedback weight (beta)"); plt.ylabel("recall@10")
plt.title("Query enrichment (pseudo-relevance feedback)");
plt.legend(fontsize=8,loc="lower left")
plt.grid(alpha=.3); plt.tight_layout(); plt.savefig(f"{FIG}/
l7_querytransform.png",dpi=130); print("saved l7_querytransform.png")

```



Listing 8 — RAGAS-style evaluation: four metrics, four failures

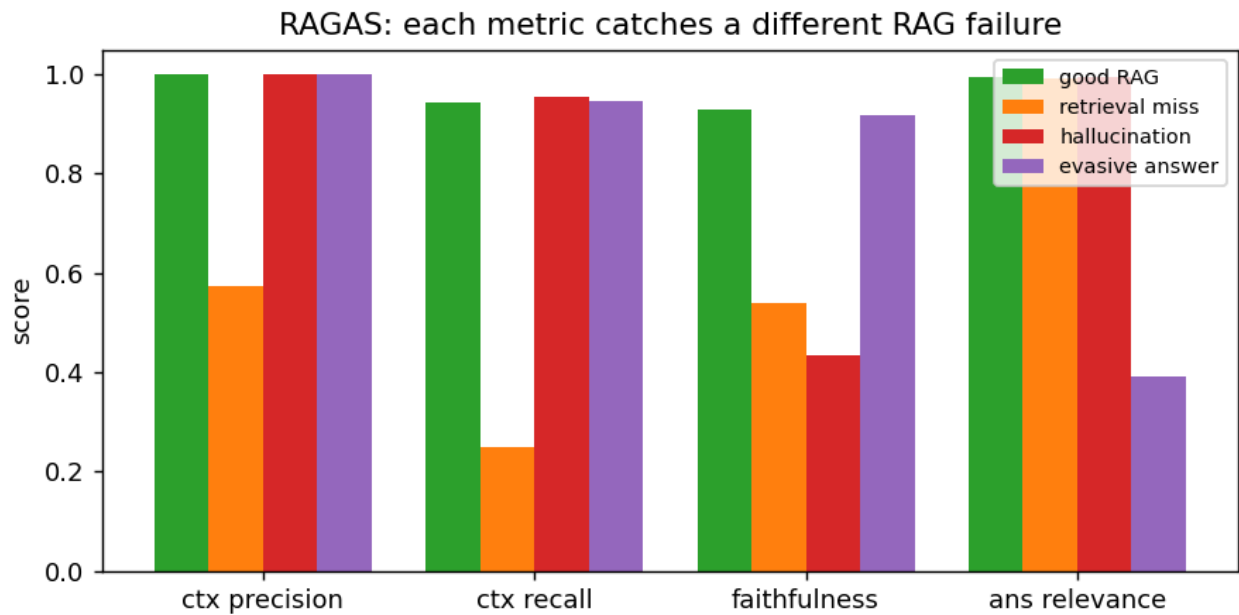
Each metric is implemented from its definition and run on controlled failure scenarios, showing that context recall, faithfulness, and answer relevance each catch a failure the others miss.

""Listing 8: RAG evaluation (RAGAS-style), an exact simulation. A RAG answer can fail in independent places, and one score cannot see all of them, so RAGAS reports FOUR: context precision (are the relevant contexts ranked high), context recall (did retrieval fetch the needed contexts), faithfulness (is every answer claim entailed by the retrieved contexts), and answer relevance (does the answer address the question). We implement each metric from its definition and run three

```

controlled
failure scenarios to show each metric isolates a DIFFERENT failure -- the reason you
need all four."""
import os, numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
FIG="figures/ch24"; os.makedirs(FIG,exist_ok=True); rng=np.random.default_rng(0)
def context_precision(ranking_rel): # ranking_rel: list of 0/1 by
    rank
    hits=0; s=0.0
    for i,r in enumerate(ranking_rel):
        if r: hits+=1; s+=hits/(i+1) # precision@rank when a
    relevant hits
    return s/max(hits,1)
def context_recall(retrieved_gold, total_gold): return retrieved_gold/total_gold
def faithfulness(claim_supported): return np.mean(claim_supported) # frac of claims
entailed
def answer_relevance(qvec, avec):
    return float(qvec@avec/((np.linalg.norm(qvec)*np.linalg.norm(avec))+1e-9))
def episode(p_gold_retrieved, rel_rank_quality, p_claim_supported, ans_align, ngold=4,
nctx=10, nclaim=5):
    # retrieval: each gold context retrieved w.p. p_gold_retrieved; rank quality
    controls ordering
    got=rng.random(ngold)<p_gold_retrieved; retrieved_gold=int(got.sum())
    rels=[1]*retrieved_gold+[0]*(nctx-retrieved_gold)
    # good rank quality -> relevant items near the top; bad -> shuffled
    order=np.array(rels,float)+rel_rank_quality*rng.random(nctx)
    ranking_rel=[rels[i] for i in np.argsort(-order)]
    cp=context_precision(ranking_rel); cr=context_recall(retrieved_gold,ngold)
    # faithfulness: if the needed context is missing, unsupported claims rise
    supp=rng.random(nclaim) < (p_claim_supported*(0.4+0.6*cr))
# grounding depends on recall
    f=faithfulness(supp)
    qv=rng.standard_normal(32); av=ans_align*qv+(1-ans_align)*rng.standard_normal(32)
    ar=answer_relevance(qv,av)
    return cp,cr,f,ar
def scenario(**kw):
    M=400; V=np.array([episode(**kw) for _ in range(M)]); return V.mean(0)
good = scenario(p_gold_retrieved=0.95, rel_rank_quality=0.2, p_claim_supported=0.97,
ans_align=0.9)
miss = scenario(p_gold_retrieved=0.25, rel_rank_quality=1.5, p_claim_supported=0.95,
ans_align=0.88)
hallu = scenario(p_gold_retrieved=0.95, rel_rank_quality=0.2, p_claim_supported=0.45,
ans_align=0.9)
evasive = scenario(p_gold_retrieved=0.95, rel_rank_quality=0.2,
p_claim_supported=0.95, ans_align=0.30)
labels=["ctx precision","ctx recall","faithfulness","ans relevance"]
for name,v in [("GOOD RAG",good),("RETRIEVAL MISS",miss),("HALLUCINATION",hallu),
("EVASIVE ANSWER",evasive)]:
    print(f"{name:15s}: "+" ".join(f"{l}={x:.2f}" for l,x in zip(labels,v)))
x=np.arange(4); w=0.2
fig,ax=plt.subplots(figsize=(7,3.6))
for k,(nm,col,v) in enumerate([("good RAG","#2ca02c",good),("retrieval
miss","#ff7f0e",miss),
("hallucination","#d62728",hallu),("evasive
answer","#9467bd",evasive)]):
    ax.bar(x+(k-1.5)*w,v,w,label=nm,color=col)
ax.set_xticks(x); ax.set_xticklabels(labels); ax.set_ylabel("score"); ax.set_ylim(0,
1.05)
ax.set_title("RAGAS: each metric catches a different RAG failure");
ax.legend(fontsize=8)
plt.tight_layout(); plt.savefig(f"{FIG}/l8_ragas.png",dpi=130); print("saved
l8_ragas.png")

```



Interview questions and answers

Q1. What is retrieval-augmented generation, and what problem does it solve?

RAG makes an LLM's knowledge non-parametric: instead of relying only on facts compressed into frozen weights, a retriever fetches relevant text from an external corpus at query time and places it in the model's context, so the model reasons over evidence it is looking at. This fixes the four failure modes of purely parametric knowledge: it is lossy (facts interpolated, not stored), stale (nothing past the training cutoff), unattributable (no source to cite), and confidently wrong (confabulation). RAG adds current, private, or citable knowledge without retraining — you re-index instead — at the cost of a retrieval step that can itself fail.

Q2. RAG vs fine-tuning: how do you decide?

Fine-tuning writes knowledge and behavior into the weights: durable, no retrieval hop at inference, and the only way to change how the model behaves (style, instruction-following, format) — but expensive to update, prone to forgetting, unattributable, and a poor fit for facts that change or need per-user access control. RAG keeps facts external: trivial to update, citable, access-controllable, small prompt — but bounded by retrieval quality and one latency hop. The rule of thumb: fine-tune the behavior, retrieve the facts. Use fine-tuning when you need new skills or a consistent style; use RAG when you need fresh, private, or verifiable knowledge; in production you usually do both.

Q3. RAG vs stuffing everything into a long context window?

Long-context is the simplest alternative and genuinely competes for small corpora — just paste the documents in. It breaks down as the corpus grows: attention cost grows with context length, relevant facts get lost in the middle (Chapter 22), per-query token cost balloons, and you still need to CHOOSE what to paste once the corpus exceeds the window — which is retrieval by another name. RAG retrieves just the relevant slice per query, keeping the prompt small and cheap. The two combine: retrieve a shortlist, then use a long window to fit more retrieved chunks than a small one could.

Q4. Why does retrieval quality dominate the whole RAG system?

Because a RAG answer is upper-bounded by what the retriever surfaces: if the needed passage is not in the retrieved set, no generation skill recovers it — the model abstains or hallucinates. Listing 1's recall/rank curve makes this concrete: with a weak embedder, recall is only 0.19 at rank 10 and 0.37 at rank 50, so most gold evidence is unretrieved even fetching fifty passages. Every downstream technique — better chunking, similarity, hybrid signals, reranking — exists to bend that recall curve upward, which is why interviews probe retrieval metrics before generation.

Q5. What are the main chunking strategies?

Fixed-size splits every N tokens (trivial but slices sentences mid-thought); recursive splits on a priority list of separators (paragraphs, then sentences, then words) so chunks respect natural boundaries (the common default); semantic starts a new chunk when consecutive sentences' embeddings diverge, making each chunk topically coherent; and parent-document / small-to-big embeds and retrieves SMALL chunks for precision but feeds the generator the larger parent passage for context. The last decouples the retrieval unit from the generation unit and is usually the highest-value upgrade.

Q6. How do you choose chunk size?

It is a precision-versus-completeness trade-off with no universal answer — it depends on your query type and embedder, which Listing 2 measures. For lexical queries (query shares surface terms with the target), recall is ~0.97 at every size, so prefer smaller chunks for lower cost and tighter grounding. For semantic queries (match by meaning), recall rises with chunk size (0.09 to 0.65 from 40 to 320 words) because a bigger chunk aggregates more weak topical signal. Larger chunks also cost more context-window budget and blur multi-topic embeddings. Tune it empirically on your own queries; a common practical answer is a few hundred tokens with overlap, plus the parent-document pattern.

Q7. Why use chunk overlap and boundary-aware splitting?

A fact can straddle a chunk boundary; if the split lands mid-sentence or mid-idea, neither resulting chunk contains the whole answer and retrieval degrades. Listing 2 measures this: at a fixed 80-word budget, a naive fixed-width cut scores 0.953 recall while sentence-aware packing scores 0.973 — two points lost purely to boundary damage. Overlap (repeating a sentence or two between adjacent chunks) is the cheap insurance: it guarantees any short span appears intact in at least one chunk, at the cost of some index bloat and duplication.

Q8. Explain the parent-document (small-to-big) retrieval pattern.

Embed and index small chunks so retrieval is precise (a focused embedding pinpoints the relevant span), but when a small chunk is retrieved, hand the generator the LARGER parent passage or a surrounding window instead of the tiny chunk. This decouples the retrieval unit from the generation unit: you get the retrieval precision of small chunks and the answer-completeness of large context simultaneously. It directly attacks the core chunking trade-off and is one of the most effective low-complexity upgrades to a naive pipeline.

Q9. What does an embedding model do, and what makes a good one for retrieval?

It maps text to a vector such that semantic similarity becomes geometric proximity, so retrieval reduces to nearest-neighbour search. For retrieval you want a bi-encoder trained with a contrastive objective (Sentence-BERT, E5, GTE, or commercial embeddings) that pulls query and relevant passage together and pushes irrelevants apart, typically with asymmetric query/passage handling and in-batch or hard negatives. Good ones are trained on retrieval data (not just raw language modeling), handle your domain and language, and fit your latency/memory budget on dimensionality. This book uses classical LSA (TF-IDF then SVD) so the mechanics stay transparent.

Q10. Compare cosine, dot product, and Euclidean distance for retrieval.

For equal-norm vectors all three induce the same ranking; they differ only when norms vary. Cosine measures the angle alone — pure semantic direction, magnitude-invariant. Dot product is $|q||p|\cos(\theta)$, so it multiplies the angle by the passage norm and can rank a long, high-norm passage above a better-aligned shorter one. Euclidean on normalized vectors is monotonic with cosine ($|a-b|^2 = 2 - 2\cos(\theta)$), so once you L2-normalize, all three agree — which is why the standard recipe is normalize embeddings and use dot product (fast) as a stand-in for cosine.

Q11. What is the dot-product length bias, and how did Listing 3 show it?

On un-normalized embeddings, ranking by raw dot product systematically favors high-norm passages regardless of topical relevance, because the norm multiplies the similarity. Listing 3 ran every query's top-10 under dot product versus cosine and correlated how often each passage was retrieved with its vector norm: +0.50 under dot product versus only +0.05 under cosine (the top decile by dot-product retrieval frequency had mean norm 0.534 versus a corpus mean of 0.385). It is a real, common bug — an index quietly biased toward long or boilerplate passages — and the fix is one line: normalize the vectors.

Q12. Which similarity metric should you use, and how do you know?

Use the metric the embedding model was TRAINED with. A model trained with a cosine objective must be scored with cosine (equivalently, normalize and use dot product); scoring it with raw dot product reintroduces the length bias of the previous answer. Some models are trained for dot product deliberately (so norm encodes something meaningful like confidence or length) and should be scored that way. The metric is a property of the model, not a free choice — check the model card, and when in doubt normalize and use cosine, which is the safe default.

Q13. What is a vector database and why do you need one?

It stores embeddings and answers nearest-neighbour queries efficiently at scale, since the naive approach — compute distance to all N vectors and sort — is $O(N)$ per query and does not scale to millions or billions of vectors. Vector databases (FAISS as a library; Chroma, Pinecone, Weaviate, Qdrant, Milvus as servers; pgvector inside Postgres) provide approximate nearest-neighbour indexes plus metadata filtering, updates, persistence, and sharding. The core value is a sublinear ANN index that trades a small, tunable amount of recall for a large speedup; the surrounding database features (filtering, hybrid, scaling) are what make it production-usable.

Q14. Explain the IVF index from scratch.

IVF (inverted file) partitions the vector space with k-means into *nlist* cells and stores each vector under its nearest centroid. At query time it computes the query's distance to the *nlist* centroids, picks the *nprobe* nearest cells, and scans ONLY the vectors in those cells — turning an *N*-vector scan into roughly an *nprobe/nlist* fraction of one. Listing 4 builds it and sweeps *nprobe* from 1 to 64 over 64 cells: recall versus exact climbs 0.56 to 1.00, and at *nprobe*=8 it reaches 0.91 recall while scanning only 13% of the corpus. *nprobe* is the recall/latency dial; higher *nprobe* means more cells scanned, more recall, more work.

Q15. Explain HNSW from scratch.

HNSW (hierarchical navigable small world) builds a proximity graph — each vector linked to its nearest neighbours — and answers a query by greedily walking the graph toward the query using a bounded priority queue of candidates (the *ef* parameter), across a hierarchy of layers so the walk starts with long hops (coarse layer) and ends with short ones (fine layer). Listing 4's from-scratch graph reaches 0.97 recall with 457 distance evaluations versus 6,020 for exhaustive search (13x fewer comparisons). Critically, the same graph with a naive single-path greedy walk collapses to 0.27 recall — the bounded-queue *ef* beam, not just the graph, is what makes it work, and *ef* is the latency/recall knob.

Q16. IVF vs HNSW: trade-offs and what you tune.

HNSW usually gives the best recall-at-latency and is the default in most vector databases, but costs more memory (storing graph edges) and handles deletes/updates poorly (the graph must be repaired). IVF is more memory-efficient and easier to update, and pairs well with product quantization (IVF-PQ) to compress vectors for billion-scale, but typically needs to scan more to match HNSW's recall. You tune *nprobe* (IVF) or *ef*-search (HNSW) at query time to move along the recall/latency curve, and build-time parameters (*nlist*, or *M* and *ef*-construction) to set the index's quality ceiling. The universal point: ANN recall is a tunable quantity, not a fixed property.

Q17. What is BM25 and why keep it in an era of dense embeddings?

BM25 is the standard sparse lexical scorer, a TF-IDF refinement with term-frequency saturation (k_1) so repeated terms have diminishing weight, and document-length normalization (b) so long documents are not unfairly favored. You keep it because it nails exact and rare-term matches — error codes, SKUs, proper nouns, code identifiers — that a dense embedder blurs into their nearest semantic neighbourhood. Dense and sparse fail in different directions, so BM25 is not a legacy baseline but a complementary signal; Listing 5 shows it actually OUTPERFORMING the weak dense retriever on exact-term queries (0.42 vs 0.26 recall@20).

Q18. What is hybrid search and how does Reciprocal Rank Fusion work?

Hybrid search runs a dense retriever and a lexical retriever (BM25) and fuses their rankings. Reciprocal Rank Fusion scores each document by the sum over retrievers of $1/(k + \text{rank})$, using RANKS rather than raw scores, so it needs no calibration between an incompatible cosine magnitude and a BM25 magnitude (k around 60 is standard). The intuition: a document ranked highly by either retriever accumulates a large reciprocal-rank score, and documents both agree on rise to the top. It is robust and parameter-light, which is why it is the default fusion method over score normalization schemes.

Q19. When does hybrid search help, and when does it not?

It helps when the two retrievers are comparably strong and their errors are decorrelated, so fusion recovers documents either alone would miss. Listing 5 shows both sides: in the exact-term regime BM25 (0.42) dominates weak dense (0.26) and equal-weight RRF lands BETWEEN them at 0.38 — fusion is no free lunch when one retriever is much better. Under vocabulary mismatch (query lacks the document's rare terms) BM25 collapses to 0.23, dense holds at 0.26, their errors decorrelate, and hybrid beats both at 0.29. The lesson: fusion is not automatic improvement — weight or tune it, and verify the retrievers are comparable.

Q20. Bi-encoder vs cross-encoder, and what is reranking?

A bi-encoder embeds query and passage SEPARATELY, so passage vectors are precomputed and indexed for fast whole-corpus search — but the two never interact, limiting precision. A cross-encoder feeds query and passage TOGETHER through the model so every query token attends to every passage token, which is far more accurate but far too slow to run over millions of passages. Reranking uses each where it fits: a bi-encoder retrieves a top-N shortlist cheaply (optimizing recall), then a cross-encoder rescores just those tens (optimizing precision). It is the standard two-stage pattern ending nearly every production RAG stack.

Q21. How much does reranking help, and what does it cost?

Listing 6 rescored each query's top-50 bi-encoder candidates with a precise query-passage scorer (the full high-dimensional TF-IDF the SVD discarded, standing in for a cross-encoder): recall@10 rose 44% (0.202 to 0.292), MRR@50 rose 66% (0.268 to 0.445), and nDCG@10 rose 65% (0.172 to 0.285). The cost is scoring N passages per query with the expensive model (N in the tens), adding latency but only on the shortlist, never the whole corpus. Given the precision gains for bounded cost, essentially every serious RAG system reranks (Cohere Rerank, bge-reranker, cross-encoder/ms-marco).

Q22. What is query transformation? Cover multi-query, HyDE, and decomposition.

Query transformation rewrites the user's query into better retrieval probes. Multi-query generates several LLM paraphrases, retrieves for each, and unions the hits — insurance against any single phrasing missing the target. HyDE (hypothetical document embeddings) asks the LLM to hallucinate an answer and retrieves with THAT, since a fake answer often sits closer in embedding space to the real answer passages than the question does. Decomposition splits a multi-hop question into sub-questions retrieved independently (the retrieval analogue of chain-of-thought). Step-back prompting abstracts to a more general question for background. All trade extra LLM calls for better recall on hard or underspecified queries.

Q23. Pseudo-relevance feedback: what is it, and why did Listing 7 show it hurting?

PRF (Rocchio) is the classical, LLM-free query expansion: retrieve once, ASSUME the top-k are relevant, and move the query vector toward their centroid, $q' = q + \text{beta} \cdot \text{mean}(\text{top-k})$. Listing 7 swept beta and recall@10 FELL monotonically (0.185 to 0.164) — feedback made it worse — because the weak first-stage top-k was polluted with off-topic passages, so the centroid dragged the query toward noise. The control proves the mechanism is sound with clean signal: expanding toward the TRUE gold centroid lifted recall to 0.41. The general lesson: every expansion method (PRF, HyDE, multi-query) is only as good as the expansion signal's quality — measure, don't assume.

Q24. What is contextual compression and why use it?

After retrieval, contextual compression filters or shrinks the passages before they reach the generator: drop chunks whose relevance score is below a threshold, extract only the query-relevant sentences within a chunk (via an LLM or a small extractive model), and re-order so the most relevant context sits where the model attends most (start or end, per lost-in-the-middle). The payoff is lower token cost and less distraction for the generator, which measurably improves faithfulness when the retrieved set is noisy. It is the post-retrieval counterpart to query transformation — clean up what you fetched before asking the model to read it.

Q25. Explain self-RAG and corrective RAG.

Both make retrieval adaptive rather than a fixed one-shot lookup. Self-RAG trains the model to emit reflection tokens that decide, per step, WHETHER retrieval is needed (many tokens need no external fact), then to critique retrieved passages for relevance and its own output for support — so retrieval and grounding become conditional and self-assessed. Corrective RAG (CRAG) adds a lightweight retrieval evaluator: if the retrieved context is judged weak or ambiguous, it triggers a corrective action — a web-search fallback, a query rewrite, or knowledge refinement — instead of feeding the generator bad context. Both target the naive pipeline's habit of blindly trusting whatever top-k returns.

Q26. What is GraphRAG and what does it solve that flat RAG cannot?

GraphRAG builds a knowledge graph of entities and relations extracted from the corpus, plus hierarchical community summaries over it, and retrieves by traversing or summarizing the graph rather than fetching independent top-k chunks. It targets GLOBAL, sensemaking questions — 'what are the main themes across these documents?' or multi-hop questions spanning several documents — that flat top-k retrieval answers poorly because no single chunk contains the answer and the relevant facts are scattered. The cost is an expensive indexing pipeline (entity/relation extraction, graph construction, summarization); the benefit is answering questions about the corpus as a whole, not just locating passages.

Q27. What are the RAGAS metrics and what does each measure?

RAGAS componentizes evaluation along the retriever/generator seam. Retrieval side: context recall (did retrieval fetch all passages needed to answer — the ceiling) and context precision (are relevant passages ranked above irrelevant ones). Generation side: faithfulness (is every answer claim entailed by the retrieved context — grounded, not hallucinated) and answer relevance (does the answer actually address the question, versus evasive or padded). They are typically computed with an LLM-as-judge decomposing answers into claims and checking entailment, which inherits judge biases (Chapter 23) and must be validated against human labels before you trust it.

Q28. Why do you need four RAG metrics instead of one end-to-end score?

Because the retriever and generator fail independently, and 'retrieval was wrong,' 'the model made something up,' and 'the model didn't answer' are different bugs with different fixes — one number cannot localize which you have. Listing 8's controlled scenarios show the orthogonality: a retrieval miss is caught by context recall crashing to 0.25 (and drags faithfulness to 0.54) while answer relevance stays 0.99; a hallucination is invisible to the context metrics (both above 0.95) and caught only by faithfulness collapsing to 0.43; an evasive answer is caught only by answer relevance falling to 0.39. To debug a RAG system you read the panel: low context recall means fix chunking/retrieval/embeddings; high context metrics but low faithfulness means constrain the generator or compress context; low answer relevance means fix the prompt or generation.